

Concurrency

Module 4

Classic Problems of Synchronization

The Bounded-Buffer Problem

- Producer-consumer problem
- The **pool consists of n buffers**, each capable of holding one item.
- The producer must not insert data when the buffer is full and the consumer must not consume data when the buffer is empty
- The producer and consumer should not operate simultaneously

The Bounded-Buffer Problem

Solution using Semaphore – Data structures used

m (mutex) – binary semaphore to acquire and release the lock

empty – a counting semaphore whose initial value is the number of buffers in the pool (i.e. initially the pool is empty)

Full – a counting semaphore whose initial value is 0

The Bounded-Buffer Problem

Solution using Semaphore

Producer

```
do {  
    wait (empty); // wait until empty>0  
                    and then decrement 'empty'  
    wait (mutex); // acquire lock  
    /* add data to buffer */  
    signal (mutex); // release lock  
    signal (full); // increment 'full'  
} while(TRUE)
```

Consumer

```
do {  
    wait (full); // wait until full>0 and  
                  then decrement 'full'  
    wait (mutex); // acquire lock  
    /* remove data from buffer */  
    signal (mutex); // release lock  
    signal (empty); // increment 'empty'  
} while(TRUE)
```

The Readers Writers Problem

- ❑ Suppose that a database is to be shared among several concurrent processes.
- ❑ Some of these processes may want only to read the database, whereas others may want to update (that is, read and write) the database.
- ❑ If two readers access the shared data simultaneously, no adverse effects will result.
- ❑ However, if a writer and some other process (either a reader or a writer) access the database simultaneously, chaos may ensue.

The Readers Writers Problem

To ensure that these difficulties do not arise, we require that the writers have exclusive access to the shared database while writing to the database.

This synchronization problem is referred to as the **readers-writers problem**

The Readers Writers Problem

Several ways to implement

1. No reader be kept waiting unless a writer has already obtained permission to use the shared object
2. Once a writer is ready, that writer perform its write as soon as possible

But these solutions may lead to starvation

The Readers Writers Problem

Implementation using Semaphore

1. **mutex**, a semaphore (initialized to 1) which is used to ensure mutual exclusion when readcount is updated.
2. **wrt**, a semaphore (initialized to 1) common to both reader and writer processes
3. **Readcount** – an integer variable (initialized to 0) that keeps track of how many processes are currently reading the object.

The Bounded-Buffer Problem

Solution using Semaphore

Writer Process

```
do {
  /* writer requests for critical
  section */

  wait(wrt);

  /* performs the write */
  // leaves the critical section

  signal(wrt);
} while(true);
```

Reader Process

```
do {
  wait (mutex);
  readcnt++; // The number of readers has now increased by 1
  if (readcnt==1)
    wait (wrt); // this ensure no writer can enter if there is even one reader
  signal (mutex); // other readers can enter while this current reader is
                  // inside the critical section
  /* current reader performs reading here */
  wait (mutex);
  readcnt--; // a reader wants to leave
  if (readcnt == 0) //no reader is left in the critical section
    signal (wrt); // writers can enter
  signal (mutex); // reader leaves
} while(true);
```

The Dining-Philosophers Problem

- problem definition
- classic synchronization problem

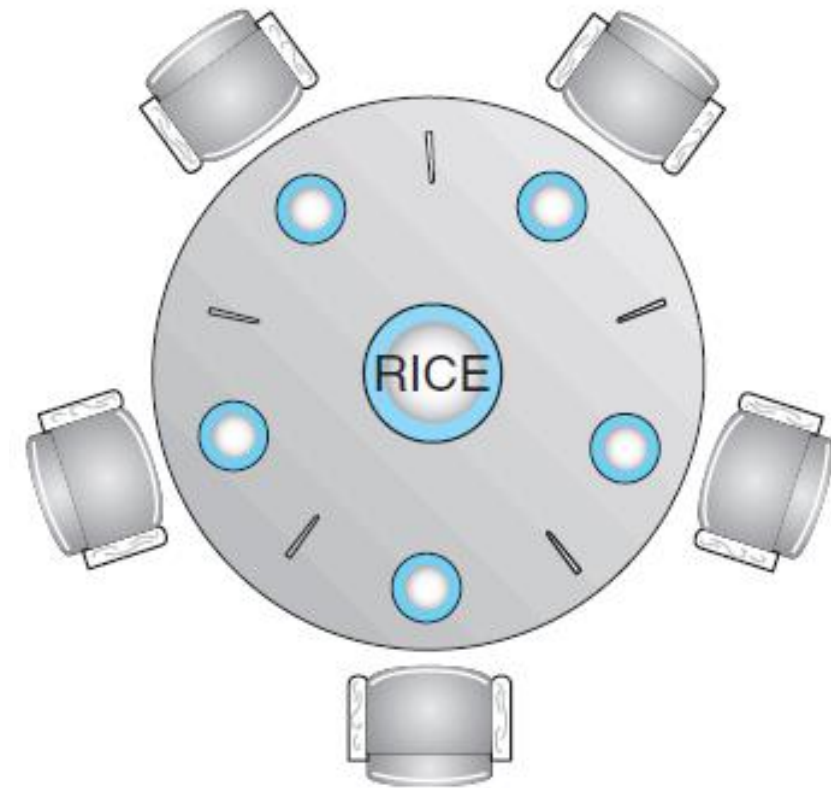


Figure: The situation of the dining philosophers.

Critical Section Problem

Data structures used

Philosopher executes a `wait()` on semaphore to grab a chopstick

Use `signal()` to release a chopstick

Five chopsticks- five semaphore i.e. **`semaphore chopstick[5];`**

Synchronization Hardware 2 – Compare and Swap

The structure of philosopher i

```
do {  
  wait (chopstick [i] );  
  wait(chopstick [ (i + 1) % 5] );  
  ....  
  // eat  
  signal(chopstick [i]);  
  signal(chopstick [(i + 1) % 5]);  
  // think  
}while (TRUE);
```

No two Philosopher will start eat at a time
Then the solution is fine.

Critical Section Problem

Several possible remedies to the deadlock problem are the following:

- Allow at most four philosophers to be sitting simultaneously at the table.
- Allow a philosopher to pick up her chopsticks only if both chopsticks are available
- Use an asymmetric solution—that is, an odd-numbered philosopher picks up first her left chopstick and then her right chopstick, whereas an even numbered philosopher picks up her right chopstick and then her left chopstick.